概念性

物模型（阿里云、三星区别）

设备端开发分类，对比，优缺点

云端（设备接入、业务逻辑开发）

开发工具两个（组成；自底向上）

vibe coding和spec coding对比表

云-端融合物联网软件开发

传统的物联网软件开发方式

物模型

物模型（Thing Model） 是对物联网设备的功能、状态及交互能力进行抽象和标准化描述的数字化模型

作用：为设备端、云端和应用端提供**统一语义与接口**，减少对底层硬件、协议和数据格式的依赖。

阿里云物联网平台、三星SmartThings

阿里云与三星SmartThings的物模型设计思路有什么不同？

- **阿里云：产品级建模**，围绕具体产品定义完整的**属性、服务和事件**。
- **SmartThings：能力组合建模**，通过组合可复用的标准**Capability**构建设备，每项能力包含属性和命令

完整的物联网应用开发涉及三端开发

设备端开发

设备端软件是运行在物联网终端设备上的软件，是连接云端服务与底层硬件的关键环节，负责设备的通信接入、业务逻辑执行、硬件控制与状态上报。

按照开发对象分类：

1. **系统软件开发**：面向设备的基础运行环境
2. **设备应用开发**：面向具体物联网业务。根据应用程序的部署和执行方式可分为：
    a. **传统编译烧录开发方式**
    b. **基于解释器运行时的开发方式**

❑ 编译烧录式开发与解释器运行时开发对比

对比维度	编译烧录式开发	解释器运行时开发
常见语言	C / C++	Python / Lua / JavaScript
执行机制	编译为设备机器代码	由运行时加载并执行脚本或字节码
应用部署	通常生成并烧录固件	通常上传脚本或代码文件
应用更新	常需要重新构建固件	可直接更新应用脚本
开发调试	编译、下载、调试	支持交互式执行与快速迭代
运行开销	通常较低	通常有额外运行时和内存开销
适用场景	资源受限、实时控制等	快速原型、教学、非硬实时业务逻辑等

云服务开发

1. **设备接入**：设备发现、控制认证、设备注册（分配ID）
2. **业务逻辑开发**：
   a. 基于IFTTT的规则驱动开发：特点：事件驱动、规则配置、低代码自动化
   b. 基于**服务编排**的业务流程开发：将**常用的功能封装为独立的服务节点**，再按照业务需求将这些节点连接起来，形成完整的执行流程。

应用端开发

1. **特点**：UI与设备端功能密切相关、可重用性高
2. **开发方式**：自动生成、可视化拖拽、大模型开发

开发工具

1. **Node-RED**：基于Node.js的开源可视化编程工具
   a. 基于流的编程模型
   b. **流**（Flow）：进行可视化编程和逻辑组织的核心单元
      i. **节点**Node：基本执行单元；输入节点（产生/接收）、处理节点（转换/判断）、输出节点（发送/显示处理结果）
      ii. **连线**Wire
      iii. **消息**Message：节点之间传递的 JavaScript 对象，承载数据
2. **TinyLink**：浙大物联网一体化开发平台
   a. 核心思路：自顶向下
   b. **传统物联网应用开发：自底向上**

基于LLM的物联网软件开发

Vibe Coding 和Spec coding对比

对比维度	Vibe Coding（氛围编程）	Spec Coding（规格驱动编程）
核心思想	通过自然语言与 AI 反复交互，逐步生成和修改程序	先定义明确的需求与设计规格，再指导 AI 按计划开发
需求描述	需求相对模糊，可在开发中不断调整	开发前明确功能需求、约束条件和验收标准
开发流程	提出需求 → 生成代码 → 运行反馈 → 反复修改	需求定义 → 技术设计 → 任务拆解 → 代码实现 → 测试验收
代码生成	AI 根据当前 Prompt 直接生成或修改代码	AI 按照设计文档和任务清单逐步实现代码
人的作用	提出想法、体验功能、反馈问题	明确需求、审核方案、确认关键决策、验收结果
AI 的作用	快速生成代码，根据反馈持续修改	整理规格、设计方案、拆解任务、编写代码并执行测试
开发过程	灵活、快速，但可能缺乏系统规划	过程更规范，开发任务和变更更易追踪
适用场景	简单应用、快速原型、探索性开发	复杂系统、多人协作、长期维护的软件项目

思考题

1. **Spec Coding相比Vibe Coding有什么优势?**
   需求与接口更明确，系统设计更一致，任务和变更可追踪，**减少反复返工，方便协作、测试和长期维护**。
2. **什么时候用Vibe Coding?**
   用于**简单应用、快速原型、探索性开发**，尤其是需求尚不明确、需要快速试验时。
3. **什么时候用Spec Coding?**
   用于**复杂系统、端云协同、多人协作、长期维护项目**，尤其是数据、接口、业务规则需要严格一致时。

IoT Coding Agent

面临的特殊挑战：硬件异构、开发环境复杂、端云跨域、真实设备验证

架构：**LLM推理决策、Context上下文、Tools工具集合、Workflow执行控制**